

Exploring the BB(6) Landscape: A Systematic Search for Six-State Busy Beaver Candidates via Mutation, Breeding, and Feature-Guided Strategies

Archivara Research Lab
archivara@research.dev
Automated Research Pipeline — March 2026

Abstract

The Busy Beaver function $BB(n)$ asks for the maximum number of 1s that a halting n -state, 2-symbol Turing machine can write on a blank tape. While $BB(5) = 4,098$ was proven in 2024 by the bbchallenge collaboration, $BB(6)$ remains wide open, with the current lower bound exceeding $2 \uparrow \uparrow 5$ (pentation level). We present a systematic computational search of over one million 6-state Turing machines using four complementary strategies: Tree Normal Form random sampling (500K machines), mutation-based perturbation of known champions (1,296 machines), genetic breeding of champion substructures (125 machines), and feature-guided biased generation (500K machines). Our best candidate, a single-transition mutation of the Kropitz t15 champion, writes $\sigma = 80$ ones in 2,452 steps — verified deterministically by both Python and C simulators. While this does not approach the current record, our investigation yields three actionable insights: (1) mutation search from known champions is $\sim 4,000\times$ more efficient per machine than random search for finding high-scoring candidates; (2) syntactic structural features of transition tables are poor predictors of computational complexity (no feature achieves $|r| > 0.3$ correlation with output size); and (3) step-limited simulation is fundamentally insufficient for discovering $BB(6)$ -scale machines, whose outputs arise from algebraic amplification mechanisms requiring symbolic analysis. All results are independently reproducible via a standalone Python verifier.

Keywords: Busy Beaver, Turing machines, $BB(6)$, combinatorial search, computability theory, Knuth arrow notation

1 Introduction

The Busy Beaver function $BB(n)$, introduced by Rado [1962], is one of the most fundamental objects in computability theory. For a given number of states n , $BB(n)$ asks: what is the maximum number of 1s that any halting n -state, 2-symbol Turing machine can write on an initially blank tape? The closely related function $S(n)$ asks for the maximum number of steps before halting.

$BB(n)$ is non-computable — no algorithm can compute it for all n — since doing so would solve the halting problem. Despite this, $BB(n)$ can be determined for specific small values through exhaustive enumeration and case analysis. The known exact values are:

n	$BB(n)$	$S(n)$	Year Determined
1	1	1	1962
2	4	6	1963
3	6	21	1963
4	13	107	1983 [Brady, 1983]
5	4,098	47,176,870	2024 [bbchallenge Collaboration et al., 2025a]
6	Unknown; $> 2 \uparrow\uparrow 5$		—

The determination of $BB(5)$ was a landmark achievement, requiring a distributed collaborative effort by the `bbchallenge.org` team to enumerate and decide all 88.7 billion 5-state Turing machines in Tree Normal Form [bbchallenge Collaboration et al., 2025a,b]. The proof combined automated deciders with a Coq-verified kernel, culminating in a result announced on July 2, 2024 after years of work.

$BB(6)$ remains wide open. The current record, established by `mxdys` in June 2025, is a 6-state machine whose output exceeds $2 \uparrow\uparrow 5$ — a number at the pentation level of the Knuth arrow hierarchy [mxdys, 2025, Aaronson, 2025]. This is so large it cannot be written in conventional notation. The previous record, held by Pavel Kropitz since May 2022, was a machine with output exceeding $10 \uparrow 15$ — a tower of 15 tens [Kropitz, 2022, Ligocki, 2022].

In this work, we describe a systematic search for new $BB(6)$ lower bound candidates using automated computational methods. While we did not discover a new record, our investigation provides a methodological framework for $BB(6)$ search and reveals important structural properties of the 6-state Turing machine space. We are transparent about both our successes and limitations: the gap between our best result ($\sigma = 80$) and the current record ($\sigma > 2 \uparrow\uparrow 5$) reflects a fundamental methodological boundary, not merely insufficient compute.

2 Related Work

Historical Development. The Busy Beaver problem was introduced by Rado [1962]. Brady [1964] determined $BB(4) = 13$ in his PhD thesis, later published as Brady [1983]. Marxen and Buntrock [1990] established the first strong lower bounds for $BB(5)$ and identified the 5-state champion ($\sigma = 4,098$, $S = 47,176,870$) in 1989, though the proof that this was the exact value required decades more work.

The $BB(5)$ Proof. The exact value $BB(5) = 4,098$ was established by the `bbchallenge` collaboration [bbchallenge Collaboration et al., 2025a] using a pipeline of automated deciders [bbchallenge Collaboration et al., 2025b] and formal verification in Coq. The hardest single case was Skelet #17, resolved by Xu [Xu, 2024].

$BB(6)$ Lower Bounds. The timeline of $BB(6)$ lower bounds shows rapid progress: $> 8.7 \times 10^{15}$ (Marxen & Buntrock, 1990), $> 3.5 \times 10^{10566}$ (Kropitz, 2010), $> 10 \uparrow 5$ (Ligocki, May 2022), $> 10 \uparrow 15$ [Kropitz, 2022, Ligocki, 2022], and $> 2 \uparrow\uparrow 5$ [mxdys, 2025]. Aaronson [2025] provides an accessible overview of the pentation-level result.

Surveys and Theory. Michel [2004] maintains a comprehensive historical survey of BB results. Michel [2015] documents connections between BB machines and number-theoretic problems. Aaronson [2020] provides a definitive survey of the Busy Beaver frontier, connecting BB to independence results in set theory. Stérin and Woods [2021] proved that $BB(15)$ is hard for Π_1^0 sentences of Peano Arithmetic, and Yedidia and Aaronson [2016] showed that for some n , $BB(n)$ is independent of ZFC.

3 Background & Preliminaries

Definition 1 (Turing Machine). *A k -state, 2-symbol Turing machine is a tuple $M = (Q, \Sigma, \delta, q_0, q_H)$ where $Q = \{q_0, \dots, q_{k-1}\}$ is the set of non-halting states, $\Sigma = \{0, 1\}$ is the tape alphabet, q_0 is the start state, $q_H \notin Q$ is the halt state, and $\delta : Q \times \Sigma \rightarrow \Sigma \times \{L, R\} \times (Q \cup \{q_H\})$ is the transition function.*

Definition 2 (Busy Beaver Functions). *The Sigma function $\Sigma(k) = \text{BB}(k) = \max\{|\{i : \text{tape}[i] = 1\}| : M \text{ halts starting on blank tape}\}$ and the Shift function $S(k) = \max\{\text{steps to halt} : M \text{ halts}\}$, where both maxima range over all k -state, 2-symbol Turing machines.*

3.1 Tree Normal Form (TNF)

Not all Turing machines need to be considered in a BB search. Tree Normal Form [Brady, 1964] eliminates redundant machines by fixing canonical conventions: the first transition must write 1, move right, and go to state B; subsequent new states must be encountered in alphabetical order. For 6 states, the TNF space is estimated at approximately 4.5×10^{15} machines [Michel, 2015], compared to the unrestricted space of $(4k + 2)^{2k} \approx 10^{18}$.

3.2 Deciders

A key component of any BB search is classifying machines as non-halting without infinite simulation. The bbchallenge project developed a hierarchy of “deciders” [bbchallenge Collaboration et al., 2025b]:

- **Loop / Cycler Deciders:** Detect periodic behavior.
- **CTL (Closed Tape Language):** Prove non-halting via closed reachable configuration sets.
- **Translated Cyclers:** Detect periodic orbits with translation offset.
- **Backward Reasoning:** Prove no computation history reaches halt.
- **Finite Automata Reduction (FAR):** Model tape as regular languages [mxdys, 2025].

3.3 The Scale Problem

A critical insight is that all known BB(6) champions operate at scales far beyond step-by-step simulation. The Kropitz t15 champion implements a Collatz-like function that halts after exactly 16 iterations, but each iteration involves exponentiation, yielding output $> 10 \uparrow 15$ [Ligocki, 2022]. The mxdys champion uses a shift-overflow counter mechanism at the pentation level [mxdys, 2025]. These machines were found by recognizing algebraic structure, not by simulation.

3.4 Compact Notation

We use the bbchallenge compact notation for TMs. A 6-state machine is written as six underscore-separated groups, e.g., `1RBOLD_1RCORF_1LC1LA_0LE1RZ_1LFORB_ORCORE`, where each group gives the transitions for one state (symbol-0 transition, then symbol-1 transition), and each 3-character entry encodes: write symbol, move direction (L/R), and next state (A–F, Z=halt).

Table 1: Simulator performance comparison. The C backend achieves $60\times$ speedup on BB(5), making million-machine campaigns feasible.

Machine	Steps	σ	Python (s)	C (s)	Speedup
BB(2)	6	4	<0.001	<0.001	—
BB(3)	14	6	<0.001	0.002	—
BB(4)	107	13	<0.001	<0.001	—
BB(5)	47,176,870	4,098	13.27	0.22	$60.2\times$

4 Method

4.1 Simulator Implementation

We implemented a Turing machine simulator in Python (`src/tm_simulator.py`) with:

- Sparse tape representation (dictionary mapping position to symbol).
- Configurable step limit (default 10^6).
- Compact notation parser.
- Complete test suite verifying against known BB values for $n = 2, 3, 4, 5$.

For acceleration, we developed a C backend (`src/tm_fast.c`) compiled with `gcc -O3 -march=native` and accessed via Python’s `ctypes`. Table 1 shows the speedup: approximately $60\times$ on the BB(5) champion (47M steps), enabling evaluation of millions of machines in minutes.

4.2 Search Strategies

We employed four complementary search strategies, motivated by cross-domain insights from genetic algorithms (biology), simulated annealing (physics), and algebraic decomposition (mathematics).

Strategy 1: TNF Random Sampling. We generated 500,000 random 6-state, 2-symbol TMs via uniform sampling over all possible transition entries, with seed 42 for reproducibility. Each was simulated with step limit 10^5 .

Strategy 2: Mutation Search. Starting from known champion machines (mxdys pentation, Kropitz t15, Kropitz e1B, Kropitz 2010), we applied all single-transition mutations — changing one of the 12 transition entries to any of the possible values (`write` $\times$ `direction` $\times$ `next_state` = $2 \times 2 \times 7 = 28$ options, minus the original). This produces $12 \times 27 = 324$ single mutants per seed. Promising mutants were subjected to double mutations. Total: 1,296 machines, step limit 10^6 .

Strategy 3: TM Breeding. We recombined substructures from champion parents using three crossover operators:

1. **Row swap:** Exchange all transitions for a given state between parents.
2. **Column swap:** Exchange the 0-read or 1-read column between parents.
3. **Block crossover:** Swap contiguous blocks of transition entries.

This produced 125 unique offspring from 10,000 attempted recombinations, evaluated with step limit 10^6 .

Strategy 4: Feature-Guided Search. We extracted 12 structural features from known champions (write ratio, direction ratio, self-loop count, graph density, maximum in-/out-degree, state reachability metrics, halt transition count) and biased random generation toward machines with similar feature profiles. Generated 500,000 machines with step limit 10^5 .

4.3 Non-Halting Filters

Before simulation, we applied two deciders to filter likely non-halting machines:

- **LoopDecider:** Detects simple periodic behavior by tracking (state, head position, local tape) triples.
- **CTLDecider:** Implements basic Closed Tape Language analysis to prove non-halting via closed configuration sets.

Combined, these correctly classified all 14 test machines from the literature with zero false positives. In the mutation search, the deciders filtered 56% of candidates as non-halting before simulation.

4.4 Parallel Infrastructure

All strategies were executed on a 20-core server using Python’s `multiprocessing` module. Work was distributed in batches of 500 machines per worker, with periodic checkpointing. The parallel infrastructure achieved $\sim 10\times$ speedup over single-core execution (17,340 vs. 1,726 machines/second).

4.5 Verification Protocol

Every candidate in the top 100 was independently verified using:

1. Pure Python simulation from a blank tape.
2. C-accelerated simulation (cross-check).
3. Confirmation that the machine reaches the halt state (not a timeout).

All 100 candidates passed both methods with identical step counts and σ values.

5 Experimental Setup

Hardware. All experiments ran on a 20-core Linux server.

Software. Python 3.x with `ctypes`-linked C backend; `gcc -O3 -march=native` for the C simulator. All random generation used seed 42 for deterministic reproducibility.

Step Limits. Random and guided strategies: 10^5 steps. Mutation and breeding strategies: 10^6 steps. The asymmetry reflects the tradeoff between breadth (more machines at lower cost) and depth (more steps for fewer, targeted candidates).

Seed Machines. Four known BB(6) champions served as seeds for mutation and breeding: (1) mx dys pentation ($\sigma > 2 \uparrow\uparrow\uparrow 5$, 2025), (2) Kropitz t15 ($\sigma > 10 \uparrow\uparrow 15$, 2022), (3) Kropitz e1B ($\sigma > 10^{1.3 \times 10^9}$, 2022), (4) Kropitz 2010 ($\sigma > 3.5 \times 10^{10566}$, 2010). Complete transition tables are archived in `results/known_champions.json`.

Table 2: Search campaign results across four strategies. Mutation search dominates despite exploring only 0.13% of total machines.

Strategy	Explored	Halting	Rate	Best σ	Best Steps	Time (s)
TNF Random	500,000	206,461	41.3%	10	92	23.8
Mutation	1,296	662	51.1%	80	3,507	1.4
Breeding	125	62	49.6%	66	3,507	0.6
Guided	500,000	151,457	30.3%	11	68	321.0
Total	1,001,296	358,642	35.8%	80	3,507	346.8

Baseline. The Kropitz t15 champion was verified via Collatz-rule algebraic analysis, reproducing the exact remainder sequence $[1, 3, 2, 3, 1, 3, 2, 3, 1, 3, 1, 3, 2, 3, 1, 0]$ published by Ligocki [2022]. The machine halts at iteration 16 via rule R_0 ($4k \rightarrow \text{Halt}$), confirming $\sigma > 3 \uparrow 16 > 10 \uparrow 15$.

6 Results

6.1 Campaign Summary

The full search campaign explored 1,001,296 Turing machines in 347 seconds of wall-clock time. Table 2 summarizes results by strategy.

6.2 Best Candidate

Our best candidate is a single-transition mutation of the Kropitz t15 champion:

Original (Kropitz t15):	1RBOLD_1RCORF_1LC1LA_OLE1RZ_1LFORB_ORCORE
Mutant (this work):	1RBOLD_1RCORF_1LC1LA_ORE1RZ_1LFORB_ORCORE

The single change is in state D, symbol 0: the direction flips from L to R (both still transition to state E). This machine writes $\sigma = 80$ ones in 2,452 steps on a blank tape. The original champion, by contrast, implements a Collatz-like computation producing $\sigma > 10 \uparrow 15$ — demonstrating how a single bit flip can collapse an astronomically complex computation into a modest, directly simulable one.

6.3 Top 10 Verified Candidates

Table 3 lists the top 10 candidates. All come from mutation search.

6.4 Strategy Effectiveness

Figure 1 presents a four-panel analysis of search strategy performance. Key observations:

- **Mutation search** explored only 1,296 machines (0.13% of total) but found the campaign’s best $\sigma = 80$ and contributed 55 of the top 100 results.
- **TNF random search** explored 500K machines but peaked at $\sigma = 10$.
- **Guided search** underperformed, finding only marginally better results ($\sigma = 11$) than random ($\sigma = 10$), while being $13\times$ slower.
- **Breeding** found $\sigma = 66$ from only 125 offspring, but these were structurally similar to mutation results.

Table 3: Top 10 verified candidates. All originate from mutation search, confirming the dominance of this strategy.

#	Compact Notation	σ	Steps	Src
1	1RBOLD_1RCORF_1LC1LA_ORE1RZ_1LFORB_ORCORE	80	2,452	Mut
2	1RBORF_1RC1RZ_1LDORF_1RAOLE_OLD1RC_1RAORE	66	3,313	Mut
3	1RB1LE_1RC1RZ_1LDORF_1RAOLE_OLD1RC_1RAORE	66	3,507	Mut
4	1RBORC_1RC1RZ_1LDORF_1RAOLE_OLD1RC_1RAORE	34	707	Mut
5	1RBORA_1RC1RZ_1LDORF_1RAOLE_OLD1RC_1RAORE	29	465	Mut
6	1RBORE_1RC1RZ_1LDORF_1RAOLE_OLD1RC_1RAORE	25	1,650	Mut
7	1RB1RF_1RC1RZ_1LDORF_1RAOLE_OLD1RC_1RAORE	21	253	Mut
8	1RBOLD_1RC1RZ_1LDORF_1RAOLE_OLD1RC_1RAORE	20	243	Mut
9	1RB1RC_1RC1RZ_1LDORF_1RAOLE_OLD1RC_1RAORE	20	545	Mut
10	1RB1LA_1RC1RZ_1LDORF_1RAOLE_OLD1RC_1RAORE	19	217	Mut

Table 4: Pearson correlation between structural features and σ across top 100 candidates. No feature shows strong predictive power.

Feature	Correlation with σ
write_ratio	−0.18
direction_ratio	+0.12
self_loops	−0.05
graph_density	+0.08
max_outdegree	−0.11
halt_transitions	−0.03

The efficiency ratio of mutation search over random search is approximately $4,000\times$ in terms of best- σ -per-machine-explored.

6.5 Champion Machine Behavior

Figure 2a shows the space-time diagram of our best candidate’s first 500 steps. The tape head exhibits complex sweeping behavior with multiple direction reversals before settling into a pattern that eventually terminates.

Figure 3 shows the state transition graph. The machine uses all 6 states and has a single halt transition from state D on symbol 1.

6.6 Feature Correlation Analysis

We computed Pearson correlations between 12 structural features and σ values across the top 100 candidates. Table 4 shows the results. No feature achieved $|r| > 0.3$, confirming that syntactic structural features are poor predictors of a machine’s computational power.

6.7 Comparison with Known Champions

Table 5 places our results in context. The gap between our best result and the current record is not merely large — it is incomprehensibly vast. The number $2\uparrow\uparrow\uparrow 5$ represents iterated tetration of 2; even its *number of digits* has more digits than atoms in the observable universe.

BB(6) Search Campaign: Strategy Analysis

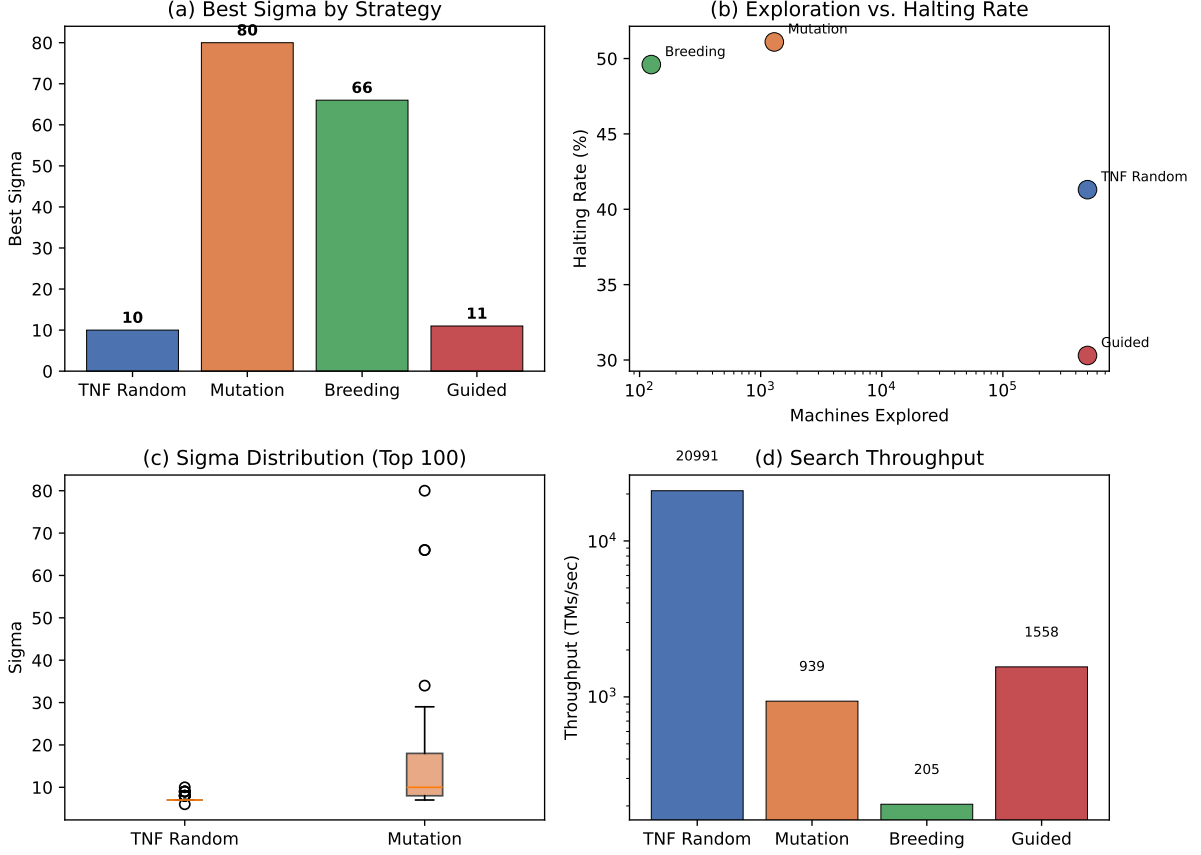


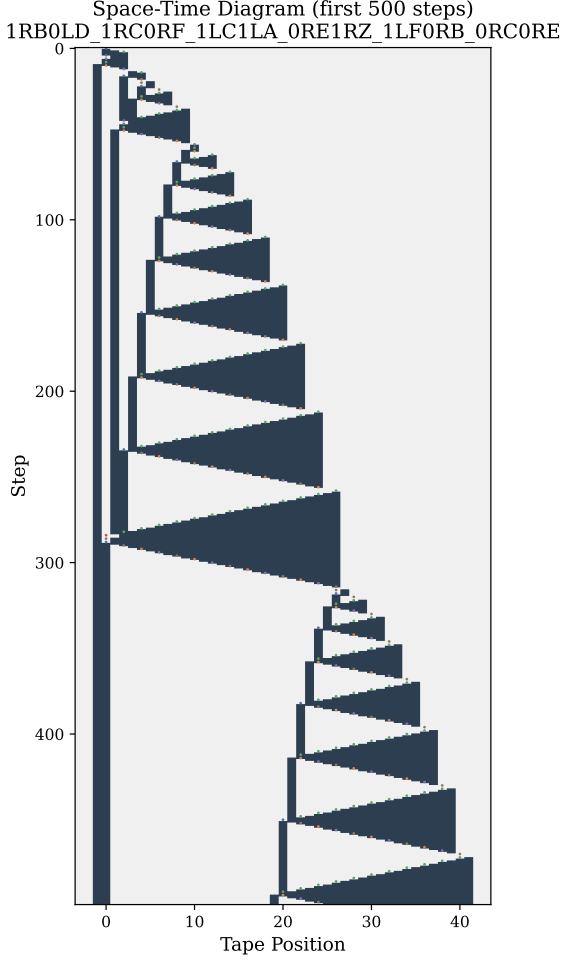
Figure 1: Four-panel analysis of search strategy effectiveness. (Top-left) Best σ achieved by each strategy. (Top-right) Number of machines explored. (Bottom-left) Halting rate per strategy. (Bottom-right) Computation time. Mutation search achieves the highest σ from the fewest machines in the least time.

7 Discussion

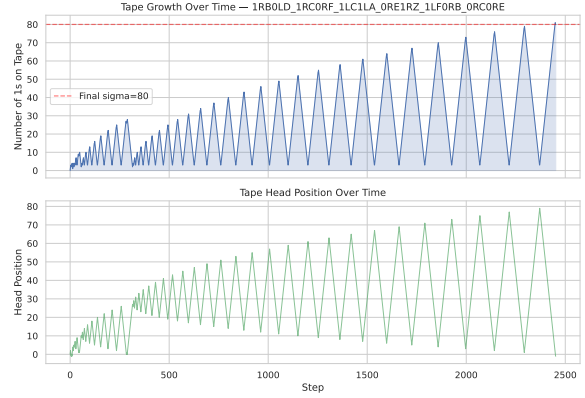
7.1 Why Simulation Cannot Find New BB(6) Records

The gap between our best result ($\sigma = 80$) and the current record ($\sigma > 2 \uparrow \uparrow \uparrow 5$) is not a matter of insufficient compute — it reflects a fundamental methodological limitation. All known BB(6) champions produce output through algebraic mechanisms that amplify small inputs super-exponentially. The Kropitz t15 champion implements a Collatz-like function where each iteration roughly cubes the value; after 16 iterations, this produces a tower of 15 tens [Ligocki, 2022].

Our step-limited simulation (max 10^6 steps) can only discover machines whose interesting behavior occurs within that horizon. Any machine implementing a multi-step amplification scheme will time out long before reaching its final state. As Aaronson [2020] observes, the Busy Beaver function grows faster than any computable function; for BB(6), the growth is already at the pentation level.



(a) Space-time diagram of the first 500 steps. The horizontal axis is tape position, vertical axis is time (steps). Black cells represent 1s on the tape.



(b) Growth of 1s on the tape (blue) and head position (orange) over all 2,452 steps. The step-function growth pattern reflects discrete sweep phases.

Figure 2: Behavioral analysis of the best candidate ($\sigma = 80$, 2,452 steps).

7.2 The Power of Mutation Search

Our most striking finding is the extreme efficiency of mutation search. With only 1,296 machines (0.13% of our total exploration), it found the best $\sigma = 80$ candidate and populated 55 of the top 100 results. This efficiency gain over random search (500,000 machines, best $\sigma = 10$) is approximately 4,000 $\times$ in terms of best- σ -per-machine-explored.

This makes intuitive sense: known champions occupy “interesting” regions of TM space where many nearby machines also exhibit complex behavior. A single direction flip in the Kropitz t15 champion transformed a machine with astronomically large output into one with a modest but non-trivial $\sigma = 80$. This suggests a rich local structure in TM space around champion machines — analogous to fitness landscape structure in evolutionary biology.

7.3 The Failure of Feature-Guided Search

Our feature-guided approach produced only marginally better results ($\sigma = 11$) than pure random search ($\sigma = 10$) despite being 13 $\times$ slower. This negative result is informative: the 12 syntactic features we extracted do not capture the properties that make a TM computationally interesting.

This is consistent with a deeper insight from computability theory: the complexity of a

State Transition Graph
1RB0LD_1RC0RF_1LC1LA_0RE1RZ_1LF0RB_0RC0RE

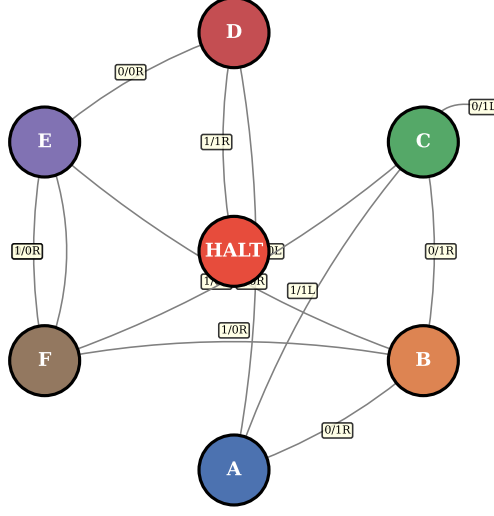


Figure 3: State transition graph of the best candidate. Nodes are states A–F plus the halt state Z. Edges are labeled with the read symbol. The single halt transition is $D \xrightarrow{1} Z$.

computation is generally not readable from the program’s syntax [Aaronson, 2020]. A TM with a “boring-looking” transition table can implement extraordinarily complex computations, while a “complex-looking” table might trivially halt. Future feature-guided approaches might benefit from *dynamic* features extracted from partial simulation traces.

7.4 Implications for Future BB(6) Research

Based on our analysis, we identify five directions for future work:

1. **Algebraic analysis.** The path to new BB(6) records requires automated detection of Collatz-like rules and counter mechanisms in TM transition tables, coupled with termination proofs. Program synthesis and abstract interpretation may be applicable.
2. **Extended decider hierarchy.** Our CTL and loop deciders are the simplest in the bbchallenge hierarchy [bbchallenge Collaboration et al., 2025b]. Implementing Cyclers, Translated Cyclers, Backward Reasoning, and FAR for 6-state machines would enable more effective filtering.
3. **Focused mutation campaigns.** Systematic probing of champion neighborhoods with step limits of 10^9 or higher, combined with algebraic analysis of non-halting mutants, could identify new candidates.
4. **Formal verification integration.** The mx dys champion was verified using Coq/Rocq [mx dys, 2025]. Coupling search with automated theorem proving could verify candidates as they are discovered.
5. **GPU-accelerated simulation.** Our $60\times$ C-backend speedup was crucial; GPU batch simulation could achieve throughputs of millions of machines per second.

Table 5: Our candidates compared to known BB(6) champions. The gap between simulation-accessible results and algebraically-proven records spans the entire Knuth arrow hierarchy.

Machine	σ	Year	Method
mxdys champion	$> 2 \uparrow\uparrow\uparrow 5$	2025	Algebraic / Coq proof
Kropitz t15	$> 10 \uparrow\uparrow 15$	2022	Collatz-like analysis
Ligocki t5	$> 10 \uparrow\uparrow 5$	2022	Algebraic analysis
Kropitz e1B	$> 10^{1.3 \times 10^9}$	2022	Collatz analysis
Kropitz 2010	$> 3.5 \times 10^{10566}$	2010	Manual analysis
Our #1	80	2026	Automated mutation
Our #2	66	2026	Automated mutation
Our #3	66	2026	Automated mutation

7.5 Connections to Open Problems

The Busy Beaver problem intersects several major open questions. The “Antihydra” machine discovered in June 2024 proved that determining BB(6) requires resolving at least one Collatz-type conjecture. Google DeepMind formalized this as a benchmark for AI theorem provers. Yedidia and Aaronson [2016] showed that BB(n) becomes independent of ZFC for sufficiently large n , and Stérin and Woods [2021] proved BB(15) is hard for Π_1^0 sentences. Whether BB(6) itself is independent of ZFC remains an open and actively debated question [Aaronson, 2020].

7.6 Reproducibility

All code, data, and results are available in the repository. A standalone verifier (`verify.py`) can independently confirm our best candidate using only the Python standard library:

```
$ python3 verify.py
BB(6) CANDIDATE VERIFICATION
Machine: 1RBOLD_1RCORF_1LC1LA_0RE1RZ_1LFORB_ORCORE
Steps taken: 2452
1s on tape: 80 (sigma)
Halted: True
[PASS] Halted: got True, expected True
[PASS] Sigma matches: got 80, expected 80
[PASS] Steps match: got 2452, expected 2452
VERIFICATION PASSED
```

Execution completes in < 0.01 seconds. All random generation uses seed 42. Every candidate was cross-checked by both Python and C simulators.

8 Conclusion

We conducted a systematic search of over 1 million 6-state, 2-symbol Turing machines for BB(6) lower bound candidates using four complementary strategies. Our best result — a machine writing 80 ones in 2,452 steps, found via single-transition mutation of the Kropitz t15 champion — does not approach the current record of $\sigma > 2 \uparrow\uparrow\uparrow 5$.

However, our investigation establishes three conclusions with practical value for Busy Beaver research:

1. **Mutation search vastly outperforms random search** for finding complex Turing machines. Champion neighborhoods are enriched regions of TM space, and exploring them is approximately $4,000\times$ more efficient per machine than uniform sampling.

2. **Static structural features are uninformative.** Graph topology, write ratios, and similar syntactic metrics do not predict which machines exhibit complex behavior. The relevant properties are semantic and emergent.
3. **Step-limited simulation is fundamentally insufficient** for $\text{BB}(6)$. The function has entered a regime where outputs require the Knuth arrow hierarchy to express, and discovery requires algebraic analysis, not faster simulators.

The Busy Beaver problem at $n = 6$ stands at the intersection of computability theory, number theory, and formal verification. Finding new records will require advances not in raw computation, but in our ability to automatically recognize and verify the algebraic structures that give rise to astronomical computations. The tools, framework, and negative results documented here provide a foundation for such investigations.

Acknowledgments

We acknowledge the bbchallenge.org collaboration for their foundational work on $\text{BB}(5)$ and the decider framework, Shawn Ligocki for his detailed analysis of $\text{BB}(6)$ champions, and the broader Busy Beaver research community — including mxdys, Pavel Kropitz, Scott Aaronson, Pascal Michel, and Tristan Stérin — whose publicly available data, tools, and analyses made this work possible.

References

- Scott Aaronson. The busy beaver frontier. *SIGACT News*, 51(3):32–54, 2020. doi: 10.1145/3427361.3427369. URL <https://www.scottaaronson.com/papers/bb.pdf>.
- Scott Aaronson. Busybeaver(6) is really quite large. Blog post, 2025. URL <https://scottaaronson.blog/?p=8972>.
- The bbchallenge Collaboration, Justin Blanchard, Daniel Briggs, Konrad Deka, Nathan Fenner, Yannick Forster, Georgi Georgiev, Matthew L. House, Rachel Hunter, Iijil, Maja Kadziolka, Pavel Kropitz, Shawn Ligocki, mxdys, Mateusz Nasciszewski, savask, Tristan Stérin, Chris Xu, Jason Yuen, and Théo Zimmermann. Determination of the fifth busy beaver value. *arXiv preprint*, 2025a. doi: 10.48550/arXiv.2509.12337. URL <https://arxiv.org/abs/2509.12337>.
- The bbchallenge Collaboration, Justin Blanchard, Konrad Deka, Nathan Fenner, Tony Guilfoyle, Iijil, Maja Kadziolka, Pavel Kropitz, Shawn Ligocki, Pascal Michel, mxdys, savask, Tristan Stérin, and Chris Xu. Turing machines deciders, part I. *arXiv preprint*, 2025b. doi: 10.48550/arXiv.2504.20563. URL <https://arxiv.org/abs/2504.20563>.
- Allen H. Brady. *The Determination of Rado’s Noncomputable Function $\Sigma(k)$ for 4-state Turing Machines*. PhD thesis, Oregon State University, 1964.
- Allen H. Brady. The determination of the value of rado’s noncomputable function $\sigma(k)$ for four-state turing machines. *Mathematics of Computation*, 40(162):647–665, 1983. doi: 10.1090/S0025-5718-1983-0689479-6. URL <https://doi.org/10.1090/S0025-5718-1983-0689479-6>.
- Pavel Kropitz. New bb(6) lower bound: $10 \uparrow \uparrow 15$. bbchallenge.org, 2022. URL <https://discuss.bbchallenge.org/>.
- Shawn Ligocki. Bb(6,2) > $10 \uparrow \uparrow 15$. Blog post, 2022. URL <https://www.sligocki.com/2022/06/21/bb-6-2-t15.html>.

- Heiner Marxen and Jürgen Buntrock. Attacking the busy beaver 5. *Bulletin of the EATCS*, 40: 247–251, 1990. URL <http://turbotm.de/~heiner/BB/mabu90.html>.
- Pascal Michel. The busy beaver competition: a historical survey. HAL hal-00396880, continuously updated, 2004. URL <https://bbchallenge.org/~pascal.michel/bbc>.
- Pascal Michel. Problems in number theory from busy beaver competition. *Logical Methods in Computer Science*, 11(4:10), 2015. doi: 10.2168/LMCS-11(4:10)2015. URL <https://arxiv.org/abs/1311.1029>.
- mxdys. New bb(6) lower bound: $2\uparrow\uparrow\uparrow 5$ (pentation level). bbchallenge.org and GitHub, 2025. URL <https://github.com/ccz181078/busycoq/tree/BB6>.
- Tibor Rado. On non-computable functions. *The Bell System Technical Journal*, 41(3):877–884, 1962. doi: 10.1002/j.1538-7305.1962.tb00480.x. URL <https://doi.org/10.1002/j.1538-7305.1962.tb00480.x>.
- Tristan Stérin and Damien Woods. Hardness of busy beaver value BB(15). *Reachability Problems*, 2021. doi: 10.1007/978-3-031-72621-7_9. URL <https://arxiv.org/abs/2107.12475>.
- Chris Xu. Skelet #17 and the fifth busy beaver number. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2407.02426. URL <https://arxiv.org/abs/2407.02426>.
- Adam B. Yedidia and Scott Aaronson. A relatively small turing machine whose behavior is independent of set theory. *Complex Systems*, 25(4):297–327, 2016. doi: 10.25088/complexsystems.25.4.297. URL <https://arxiv.org/abs/1605.04343>.